

PhyloPower — Supplementary Help Document

End-to-end tutorial: installation, gene and protein workflows, outputs, and tree perturbation

PhyloPower tutorial

End-to-end walkthrough of the two manuscript-aligned workflows for phylogeny-aware power and minimum-sample-size analysis of meta-omics beta-diversity studies:

```
| Workflow | Synthetic generator | Distance |  
|---|---|---|  
| `gene` | PCAM | Gemelli phylogenetic RPCA |  
| `protein` | MDC-TF-MC | PhyloFunc |
```

Both workflows build a synthetic raw-feature pool from a pilot dataset, estimate PERMANOVA power by a non-parametric bootstrap at several candidate per-group sample sizes, fit a monotone power curve over the realized effect size (ω^2), and report the smallest per-group `n` whose fitted power reaches the target (default 0.8).

1. Environment and installation

The **protein** workflow only needs the PyPI dependencies:

```
```bash  
python -m pip install .
```
```

The **gene** workflow additionally requires a QIIME 2 environment with Gemelli and `biom-format` (the validated environment is `qiime2-metagenome-2024.10`), because PCAM calls the Gemelli Python API in-process. Install and run PhyloPower **inside** that environment:

```
```bash  
conda activate qiime2-metagenome-2024.10
python -m pip install .
```
```

> **Warning:** without Gemelli/biom the `gene` workflow stops with a clear > error; the `protein` workflow and `--help` work in any environment with the > PyPI dependencies only.

After installation, the CLI is available both as `phylopower` and as `python -m phylopower`. A single-file runner (`phylopower\_cli.py`) that embeds all project-local code is also shipped for environments where installing the package is inconvenient.

```
# Sanity check: the CLI loads and exposes both workflows.  
!python -m phylopower --help
```

2. Gene workflow (PCAM + Gemelli)

The bundled demo dataset (``phylopower/datagene/``: feature table, rooted tree, taxonomy, group map) is used automatically when no input paths are given.

The quick/default Monte Carlo settings are ``--boot-number 200`` bootstrap iterations and ``--permutations 199`` permutations per PERMANOVA (scikit-bio applies the ``+1`` correction internally, so the p-value denominator is 200). For publication-grade estimates, increase both (see ``REPRODUCIBILITY.md``).

Run this cell **inside the QIIME 2 environment** (see section 1).

```
# Gene workflow on the bundled demo data (requires the QIIME 2 / Gemelli environment).
!python -m phylopower gene \
  --target-power 0.80 \
  --boot-number 200 --permutations 199 \
  --out tutorial_gene_result
```

3. Protein workflow (MDC-TF-MC + PhyloFunc)

The bundled demo dataset (``phylopower/datapro/``: Taxon–Function table, rooted tree, group map) is likewise used by default. No QIIME 2 environment is needed for this workflow.

```
# Protein workflow on the bundled demo data (no QIIME 2 needed).
!python -m phylopower protein \
  --target-power 0.80 \
  --boot-number 200 --permutations 199 \
  --out tutorial_protein_result
```

The same analysis is available as a Python call:

```
from phylopower import compute_protein_min_sample_size, demo_path

result = compute_protein_min_sample_size(
    table=demo_path("datapro", "protein_taxon_function_cleaned.csv"),
    tree=demo_path("datapro", "rooted-tree.nwk"),
    group=demo_path("datapro", "group.csv"),
    target_power=0.80,
    boot_number=200,
    permutations=199,
    out="tutorial_protein_result_api",
)
print(result["summary"]["minimum_n_per_group"])
```

4. Reading the outputs

Each run writes four files into ``--out``:

- `summary.json` — the headline result: ``minimum_n_per_group`` (the recommended per-group sample size, or ``null`` if the target power was not reached within ``--min-n, --max-n``), the resolved ``target_omega2``, the Monte Carlo settings, and the search configuration.
- `power_by_sample_size.csv` — one row per evaluated candidate ``n``: the fitted power at the target ω^2 , the curve-fit status, and

whether the candidate qualified.

- `scenario_metrics_by_sample_size.csv` — the raw evidence behind the fits: per effect-scenario and per `n`, the realized ω^2 of the synthetic pool (`true_omega2`) and the bootstrap power.
- `sample_size_decision.png` — the power-versus-`n` decision plot.

```
import json
import pandas as pd

summary = json.load(open("tutorial_protein_result/summary.json"))
print("minimum n per group:", summary["minimum_n_per_group"])
print("resolved target omega^2:", summary["target_omega2"])

power = pd.read_csv("tutorial_protein_result/power_by_sample_size.csv")
power[["n_per_group", "fitted_power_at_target_omega2", "qualifies"]].head()

metrics = pd.read_csv("tutorial_protein_result/scenario_metrics_by_sample_size.csv")
metrics[["n_per_group", "true_omega2", "power"]].head()
```

5. Optional: phylogenetic-uncertainty sensitivity analysis

Both workflows accept tree-perturbation options that propagate phylogenetic uncertainty into the distance computation. Nearest-neighbor interchanges (NNI) are applied first, then branch-length jitter:

- `--tree-nni-prob 0.1` — swap the sub-clades of each eligible internal node with probability 0.1;
- `--tree-jitter-sigma 0.1` — multiply every branch length by `exp(0.1 * epsilon)` with `epsilon ~ N(0, 1)`;
- `--tree-support-threshold 80` — restrict NNI to nodes whose support value is below 80 (when support values are stored as node names).

Given the same `--random-seed`, a perturbed run is bit-for-bit reproducible.

```
# Example: protein workflow under mild tree perturbation.
!python -m phylopower protein \
  --target-power 0.80 \
  --boot-number 200 --permutations 199 \
  --tree-nni-prob 0.1 --tree-jitter-sigma 0.1 \
  --out tutorial_protein_treepert
```

6. Running the gene workflow against a QIIME 2 environment

PCAM calls the Gemelli Python API `**in-process**`, so the recommended way to run the gene workflow is to activate the QIIME 2 environment first:

```
``bash
conda activate qiime2-metagenome-2024.10
python -m phylopower gene --target-power 0.80 --out gene_result
``
```

The `--qiime-env` option (default `qiime2-metagenome-2024.10`) records which

environment the run targets, but it is **not** a substitute for activating that environment — merely selecting it with `--qiime-env` from a non-QIIME shell will still fail at the Gemelli import with an explanatory error message.

All stochastic steps (pool generation, bootstrap resampling, PERMANOVA permutations, tree perturbation) are derived from `--random-seed` (default `20260614`), so a recorded command line plus the input-file checksums fully determines the outputs.